

Dictionnaire de classe

Table des matières

1 – Classe CApi.....	2
1.1 – Attributs de CApi.....	2
1.2 – Méthodes de CApi.....	2
2 - Classe CJsonMeteo.....	3
2.1 - Attribut de CJsonMeteo.....	3
3 – Classe CInfoMeteo.....	3
3.1 - Attribut de CInfoMeteo.....	3
3.2 - Méthodes de CInfoMeteo.....	3
4 - Classe CTemperatureAir.....	4
4.1 - Méthodes de CTemperatureAir.....	4
5 - Classe CHygrometrie.....	5
5.1 - Méthodes de CHygrometrie.....	5
6 - Classe CPluviometrie.....	5
6.1 - Méthodes de CPluviometrie.....	5
7 - Classe CVitesseVent.....	5
7.1 - Méthodes de CVitesseVent.....	5
8 - Classe CDirectionVent.....	5
8.1 - Méthodes de CDirectionVent.....	5
9 - Classe CEnsoleillement.....	5
9.1 - Méthodes de CEnsoleillement.....	6
10 - Classe CEvapotranspiration.....	6
10.1 - Attributs de CEvapotranspiration.....	6
10.2 - Méthodes de CEvapotranspiration.....	6
11 – Classe ApplicationHippodrome.....	8
11.1 - Attributs de ApplicationHippodrome.....	8
11.1.1 - Attributs TabPage, TabControl et ToolTip.....	8
11.1.2 - Attributs CApi, CInfoMeteo, CInfoTerrain et CARrosage.....	8
11.1.3 - Label.....	9
11.1.4 - Attributs Autre.....	9

1 – Classe CApi

Cette classe a pour objectif de pouvoir communiquer à une API.

1.1 – Attributs de CApi

- **mStrAddrAPI : string**

Sa valeur est l'adresse IPv4 de l'API. Cet attribut a un set et un get

- **mStrPortAPI : string**

Sa valeur est le port de l'API. Cet attribut a un set et un get

- **mStrQuery : string**

Sa valeur sert à formuler la commande envers l'API et donc passer en paramètres différentes valeurs. Cet attribut a une méthode permettant de formuler la query (pseudo set) et un get

- **mObjHttpClient : HttpClient**

Classe permettant de faire des envois HTTP sur des site (GET, POST, ...)

1.2 – Méthodes de CApi

+ **CApi()**

Constructeur par défaut

+ **mStrFunGetmStrAddrAPI() : string**

get de mStrAddrAPI

+ **mVFunSetmStrAddrAPI(strAdresse : string) : void**

set de mStrAddrAPI

+ **mStrFunGetmStrPortAPI() : string**

get de mStrPortAPI

+ **mVFunSetmStrPortAPI(strPort : string) : void**

set de mStrPortAPI

+ **MVFunCreateGETQuery (**
 strDataType : string,
 dtmDateDeDebut : DateTime,
 dtmDateDeFin : DateTime
) : void

1. strDataType : Type de données demandé (TemperatureAir, HygrometrieMoyenne, ...). Tout objet exploitant cette classe ont un attribut mStrDataType permettant de la passer en paramètre
2. dtmDateDeDebut : La date souhaité de la donnée la moins récente.
3. dtmDateDeFin : La date souhaité de la donnée la plus récente.

Cette méthode permet de formuler la demande de données en utilisant les paramètres passé dans la méthode, mais aussi l'attribut privé mStrAddrAPI et mStrPortAPI. Cette formulation est entrée dans l'attribut mStrQuery.

+ **mStrFunGetJsonQuery() : String**

Cette méthode permet de faire une requête HTTP à l'aide de son attribut mObjHttpClient. Elle envoie le contenu de l'attribut mStrQuery. Le renvoi de cette méthode est la réponse renvoyé par l'API sous forme de string. Le renvoi peut être converti en json.

2 - Classe CJsonMeteo

Classe entité qui va servir de stockage pour les réponses de la classe CApi en ce qui concerne la météo. La classe utilise l'espace de nom System.Text.Json. Cet espace permet de convertir un string en pseudo json, permettant alors faire les mêmes commandes que l'on ferait à un json normal (exploitation de la valeur à travers la clef, ...)

Cette classe pourra stocker les json ayant comme clefs "valeur" et "date".

Il n'y a pas besoin de champs additionnel comme la localisation car un type de données est prélevé par seulement un capteur. Le type de donnée envoyant n'est pas précisé dans le json mais est implicitement connaissable par le nom de l'objet et de la classe l'exploitant.

2.1 - Attribut de CJsonMeteo

+ mFltValeur : float

Attribut qui représente la clef "valeur".

+ mDtmDate : DateTime

Attribut qui représente la clef "date".

3 – Classe CInfoMeteo

La classe CInfoMeteo sert à demander et traiter des informations sur la météo. Pour cela, il va pouvoir exploiter un objet CApi en association pour faire ses demandes à l'API. Mais aussi la classe CJsonMeteo en composition.

3.1 - Attribut de CInfoMeteo

mStrDataType : string

Il contient le type de donnée qu'il traite (TemperatureAir, Hygrométrie, ...), il permet aussi de recevoir le bon type de données de l'API à travers un objet CAPI. Cet attribut a un get mais n'a pas de set. Cependant, il est modifié dans les constructeurs de CInfoMeteo ainsi que ces héritiers

mObjCJsonMeteoRecent : CJsonMeteo

Il contient la donnée la plus récente données par l'API à travers toute les demande

(Si l'on demande des données ayant comme date une antérieur à cet attribut, alors il ne sera pas changé, et inversement). Il a un get, il n'a pas de set manuel.

mLstObjCJsonMeteo : list CJsonMeteo

Liste qui va stocker la réponse de l'objet CAPI concernant la météo. Il a un get et un set. Le set accepte en paramètre un string et non une liste CJsonMeteo

mObjCApi : CApi

Attribut référençant un objet CApi, cet association va permettre d'exploiter un objet CApi.

3.2 - Méthodes de CInfoMeteo

+ CInfoMeteo()

Constructeur de classe par défaut. mStrDataType prend la valeur "NonDefini".

+ CInfoMeteo(objCAPI : CApi)

Constructeur de classe. mStrDataType prend la valeur "NonDefini". Association à l'objet CAPI passé en paramètre.

+ mVFunSetAssoCApi(objCApi : CApi) : void

Methode qui fait l'association avec un objet CApi. mObjCApi fait référence à objCApi passé en paramètre.

+ mVFunUpdatemLstObjCJsonMeteo(DateTime dtmDateDeDebut, DateTime dtmDateDeFin, bool boolForcerUpdate = false) : void

dtmDateDeDebut : Date de la donnée la plus ancienne souhaitée (exemple : 09/09/2025)

dtmDateDeFin : Date de la donnée la plus récente souhaitée (exemple : 13/09/2025)

boolForcerUpdate : Mettre True pour forcer la mise à jour des données, mis à faux par défaut

Méthode qui demande à l'API les données dans la période entre dtmDateDeDebut et dtmDateDeFin pour les stocker dans une liste de CJsonMeteo (qui est un attribut de classe)

1. La méthode vérifie si son parent a déjà toute les données (sauf si boolForcerUpdate est true)
2. La méthode transmet les paramètres à CApi pour former et envoyer la requête
3. Enfin, elle passe en plus en paramètre son mstrDataType pour préciser quel est son type de données
 1. mstrDataType n'a pas de set, elle est seulement modifiée dans les constructeurs des hérités de CInfoMeteo
4. Ensuite, elle récupère le retour, c'est un json sous forme de string
5. La méthode mVFunSetmLstObjCJsonMeteo(strDonneAConvertir : string) est appelée pour convertir le string en données exploitable

+ mLstCJsonMeteoFunDemandeMoyenne(DateTime dtmDateDeDebut, DateTime dtmDateDeFin) : void

dtmDateDeDebut : Date de la donnée la plus ancienne souhaitée (exemple : 09/09/2025)

dtmDateDeFin : Date de la donnée la plus récente souhaitée (exemple : 13/09/2025)

Méthode qui demande la moyenne d'une donnée journalière sur la période entre dtmDateDeDebut et dtmDateDeFin

1. La méthode transmet les paramètres à un objet CApi pour former et envoyer la requête
2. La méthode passe en plus son attribut mStrDataType concaténé avec « Moyenne »
3. Ensuite, la méthode récupère le retour sous forme de string
4. Elle deserialise se string en une liste d'objet CJsonMeteo puis la retourne

+ mVFunSetmLstObjCJsonMeteo(string strDonneAConvertir) : void

strDonneAConvertir : Données type string à convertir en liste CJsonMeteo

Méthode qui convertit un string en liste CjsonMeteo, elle est ensuite stockée dans l'attribut mLstObjCJsonMeteo.

Une vérification est faite pour savoir si il y a une donnée plus récente que mObjCJsonMeteoValeurRecente, si c'est le cas, la valeur est remplacée

mVFunVerifiermLstObjCJsonMeteo() : virtual void

Méthode appelant mVFunVerifiermLstObjCJsonMeteo(float, float, short)

Elle passe en paramètre -50, 2000 et 200.

Les héritiers de CinfoMétéo pourront réécrire cette méthode afin de passer d'autres valeurs en paramètre

mVFunVerifiermLstObjCJsonMeteo(fltValMin : float, fltValMax : float, shtEcartHoraire : short) : virtual void

protected void mVFunVerifiermLstObjCJsonMeteo(float fltValMin, float fltValMax, short shtEcartHoraire)

fltValMin : Valeur minimale acceptée dans mFltValeur

fltValMax : Valeur maximale acceptée dans mFltValeur

shtEcartHoraire : Ecart autorisé d'une valeur comparée à ces valeurs adjacentes enregistrées l'heure d'avant et après

La méthode vérifie la logique des données, c'est à dire qu'elle doit repérer au mieux les valeurs anormales

La première manière de repérer une valeur anormale est si celle ci a une valeur trop grande ou trop petite

On doit donc vérifier que toute les valeurs soient entre fltValMin et fltValMax

Si ce n'est pas le cas, elle prend la valeur la plus proche entre fltValMin et fltValMax

La deuxième manière est d'observer la continuité des valeurs les unes après les autres

Si on a un pic soudain dans les valeurs, ça signifie surement que celle si est éronnée

Une valeur anormale peut être remarquée par ces valeurs environnantes, elle est anormalement haute ou basse comparée au reste

L'idéal est de pouvoir la comparer avec avec celle de l'heure d'avant et d'après

Cependant, il faut d'abord vérifier si la liste est assez longue pour le permettre

Il faut une longueurde au moins 3 éléments

Si jamais la valeur est en effet anormale, alors elle devient la moyenne entre la valeur de l'heure d'avant et celle d'après

4 - Classe CTemperatureAir

Classe héritant de CInfoMeteo. Elle va pouvoir demander les données concernant la température de l'air.

4.1 - Méthodes de CTemperatureAir

+ CTemperatureAir() :

Constructeur par défaut. mStrDataType prend la valeur de "TemperatureAir"

+ CTemperatureAir(objCApi : CApi) :

Constructeur. mStrDataType prend la valeur de "TemperatureAir". Appelle le constructeur de CInfoMeteo en passant en paramètre objCAPI.

mVFunVerifiermLstObjCJsonMeteo() : override void

Méthode qui réécrit InfoMeteo.mVFunVerifiermLstObjCJsonMeteo()

Elle exécute la méthode mVFunVerifiermLstObjCJsonMeteo(float, float, short) en passant en paramètre -20, 80 et 3.

5 - Classe CHygrometrie

Classe héritant de CInfoMeteo. Elle va pouvoir demander les données concernant l'hygrométrie, ou le taux d'humidité dans l'air.

5.1 - Méthodes de CHygrometrie

+ CHygrometrie() :

Constructeur par défaut. mStrDataType prend la valeur de "Hygrometrie"

+ CHygrometrie(objCAPI : CAPI) :

Constructeur. mStrDataType prend la valeur de "CHygrometrie". Appelle le constructeur de CInfoMeteo en passant en paramètre objCAPI.

mVFunVerifiermLstObjCJsonMeteo() : override void

Méthode qui réécrit InfoMeteo.mVFunVerifiermLstObjCJsonMeteo()

Elle exécute la méthode mVFunVerifiermLstObjCJsonMeteo(float, float, short) en passant en paramètre 0, 100 et 4

6 - Classe CPluviometrie

Classe héritant de CInfoMeteo. Elle va pouvoir demander les données concernant la pluviométrie.

6.1 - Méthodes de CPluviometrie

+ CPluviometrie() :

Constructeur par défaut. mStrDataType prend la valeur de "Pluviometrie"

+ CPluviometrie(objCAPI : CAPI) :

Constructeur. mStrDataType prend la valeur de "Pluviometrie". Appelle le constructeur de CInfoMeteo en passant en paramètre objCAPI.

mVFunVerifiermLstObjCJsonMeteo() : override void

Méthode qui réécrit InfoMeteo.mVFunVerifiermLstObjCJsonMeteo()

Elle exécute la méthode mVFunVerifiermLstObjCJsonMeteo(float, float, short) en passant en paramètre 0, 1000 et 1000

7 - Classe CVitesseVent

Classe héritant de CInfoMeteo. Elle va pouvoir demander les données concernant la vitesse du vent.

7.1 - Méthodes de CVitesseVent

+ CVitesseVent() :

Constructeur par défaut. mStrDataType prend la valeur de "VitesseVent"

+ CVitesseVent(objCAPI : CAPI) :

Constructeur. mStrDataType prend la valeur de "VitesseVent". Appelle le constructeur de CInfoMeteo en passant en paramètre objCAPI.

mVFunVerifiermLstObjCJsonMeteo() : override void

Méthode qui réécrit InfoMeteo.mVFunVerifiermLstObjCJsonMeteo()

Elle exécute la méthode mVFunVerifiermLstObjCJsonMeteo(float, float, short) en passant en paramètre 0, 80 et 40

8 - Classe CDirectionVent

Classe héritant de CInfoMeteo. Elle va pouvoir demander les données concernant la direction du vent.

8.1 - Méthodes de CDirectionVent

+ CDirectionVent() :

Constructeur par défaut. mStrDataType prend la valeur de "DirectionVent"

+ CDirectionVent(objCAPI : CAPI) :

Constructeur. mStrDataType prend la valeur de "DirectionVent". Appelle le constructeur de CInfoMeteo en passant en paramètre objCAPI.

9 - Classe CEnsoleillement

Classe héritant de CInfoMeteo. Elle va pouvoir demander les données concernant l'ensoleillement.

9.1 - Méthodes de CEnsoleillement

+ CEnsoleillement() :

Constructeur par défaut. mStrDataType prend la valeur de "Ensoleillement"

+ CEnsoleillement(objCAPI : CAPI) :

Constructeur. mStrDataType prend la valeur de "Ensoleillement". Appelle le constructeur de CInfoMeteo en passant en paramètre objCAPI.

mVFunVerifiermLstObjCJsonMeteo() : override void

Méthode qui réécrit InfoMeteo.mVFunVerifiermLstObjCJsonMeteo()

Elle exécute la méthode mVFunVerifiermLstObjCJsonMeteo(float, float, short) en passant en paramètre 200, 2000 et 500

10 - Classe CEvapotranspiration

Classe héritant de CInfoMeteo. Elle va pouvoir calculer l'évapotranspiration grâce à la formule de Penman Monteith. Cette formule requiert différent paramètre comme l'ensoleillement, la température de l'air, l'hygrométrie, la vitesse du vent, l'albédo et l'altitude. Cette classe va donc avoir des associations avec les objets CEnsoleillement, CTemperatureAir, CHygrométrie et CVitesseVent

10.1 - Attributs de CEvapotranspiration

- mFltAlbedo : float

Paramètre de l'albédo contenu entre 1 et 0. Cet attribut a un get et un set.

- mFltAltitude : float

Paramètre de l'altitude. Cet attribut a un get et un set.

- mObjCTemperature : CTemperature

Permet de faire une association avec un objet CTemperatureAir. Cet attribut a un get et un set (mVFunSetAssoCTemperatureAir)

- mObjCHygrometrie : CHygrometrie

Permet de faire une association avec un objet CHygrometrie. Cet attribut a un get et un set (mVFunSetAssoCHygrometrie)

- mObjCEnsoleillement : CEnsoleillement

Permet de faire une association avec un objet CEnsoleillement. Cet attribut a un get et un set (mVFunSetAssoCEnsoleillement)

- mObjCVitesseVent : CVitesseVent

Permet de faire une association avec un objet CVitesseVent. Cet attribut a un get et un set (mVFunSetAssoCVitesseVent)

- mLstObjCJsonMeteoMoyenneEnsoleillement : <<List>> CJsonMeteo

Liste CJsonMeteo qui va stocker les moyennes quotidiennes sur l'ensoleillement

- mLstObjCJsonMeteoMoyenneHygrometrie : <<List>> CJsonMeteo

Liste CJsonMeteo qui va stocker les moyennes quotidiennes sur l'hygrométrie

- mLstObjCJsonMeteoMoyenneTemperatureAir : <<List>> CJsonMeteo

Liste CJsonMeteo qui va stocker les moyennes quotidiennes sur la température de l'air

- mLstObjCJsonMeteoMoyenneVitesseVent : <<List>> CJsonMeteo

Liste CJsonMeteo qui va stocker les moyennes quotidiennes sur la vitesse du vent

10.2 - Méthodes de CEvapotranspiration

+ CEvapotranspiration()

Constructeur par défaut. mStrDataType prend la valeur de « Evapotranspiration »

+ CEvapotranspiration(objCAPI : CAPI)

Constructeur. Fait l'association avec l'objet CAPI. mStrDataType prend la valeur de « Evapotranspiration »

+ CEvapotranspiration(objCAPI : CAPI, objCTemperatureAir : CTemperatureAir, objCHygrometrie : CHygrometrie, objCEnsoleillement : CEnsoleillement, objCVitesseVent : CVitesseVent) :

Constructeur. Fait l'association avec les différents objets passé en paramètres. mSrDataType prend la valeur de « Evapotranspiration »

+ mVFunCalculEvapotranspiration (dtmDateDeDebut : DateTime, dtmDateDeFin : DateTime, boolForcerUpdate = false : bool) : void

1. dtmDateDeDebut : La date souhaité de la donnée la moins récente.
2. dtmDateDeFin : La date souhaité de la donnée la plus récente.
3. boolForcerUpdate : Choisit si l'on force le calcul même si l'on a déjà les données

Calcul l'évapotranspiration en prenant la moyenne des valeurs des objets associés (hors CApi) entre les deux dates. L'évapotranspiration quotidiennes est calculée et stockée dans mLstObjCJsonMeteo. Le calcul de l'évapotranspiration suit la formule de Penman-Monteith

+ mFltFunCalculEvapotranspiration(fltMoyenneTemperatureAir : float, fltMoyenneHygrometrie : float, fltMoyenneEnsoleillement :float, fltMoyenneVitesseVent : float) : float

Effectue le calcul de l'évapotranspiration avec la formule de Penman Monteith et renvoi la valeur

+ mVFunSetmLstObjCJsonMeteoMoyenneEnsoleillement(strDonneesAConvertir : string) : void

Vérifie si les mots « valeur » et « date » sont dans le paramètre, puis convertit le string en liste CJsonMeteo et le met dans l'attribut mLstObjCJsonMeteoMoyenneEnsoleillement. Enfin il vérifie si les valeurs sont cohérentes avec

mVFunVerifiermLstObjCJsonMeteoMoyenne(this.mLstObjCJsonMeteoMoyenneEnsoleillement, 0F, 2000F, 1000);

+ mVFunSetmLstObjCJsonMeteoMoyenneHygrometrie(strDonneesAConvertir : string): void

Vérifie si les mots « valeur » et « date » sont dans le paramètre, puis convertit le string en liste CJsonMeteo et le met dans l'attribut mLstObjCJsonMeteoMoyenneHygrometrie. Enfin il vérifie si les valeurs sont cohérentes avec

mVFunVerifiermLstObjCJsonMeteoMoyenne(this.mLstObjCJsonMeteoMoyenneEnsoleillement, 0F, 100F, 4);

+ mVFunSetmLstObjCJsonMeteoMoyenneTemperatureAir(strDonneesAConvertir : string) : void

Vérifie si les mots « valeur » et « date » sont dans le paramètre, puis convertit le string en liste CJsonMeteo et le met dans l'attribut mLstObjCJsonMeteoMoyenneTemperatureAir. Enfin il vérifie si les valeurs sont cohérentes avec

mVFunVerifiermLstObjCJsonMeteoMoyenne(this.mLstObjCJsonMeteoMoyenneEnsoleillement, -20F, 80F, 3);

+ mVFunSetmLstObjCJsonMeteoMoyenneVitesseVent(strDonneesAConvertir : string) : void

Vérifie si les mots « valeur » et « date » sont dans le paramètre, puis convertit le string en liste CJsonMeteo et le met dans l'attribut mLstObjCJsonMeteoMoyenneVitesseVent. Enfin il vérifie si les valeurs sont cohérentes avec

mVFunVerifiermLstObjCJsonMeteoMoyenne(this.mLstObjCJsonMeteoMoyenneEnsoleillement, 0F, 80F, 80);

+ mVFunUpdatemLstObjCJsonMeteoMoyenneEnsoleillement(dtmDateDeDebut : DateTime, dtmDateDeFin : DateTime, boolForcerUpdate = false : bool) : void

Met à jour mLstObjCJsonMeteoMoyenneEnsoleillement si il n'a pas déjà les données dans l'attributs (sauf si boolForcerUpdate est à true)

+ mVFunUpdatemLstObjCJsonMeteoMoyenneHygrometrie(dtmDateDeDebut : DateTime, dtmDateDeFin : DateTime, boolForcerUpdate = false : bool) : void

Met à jour mLstObjCJsonMeteoMoyenneHygrometrie si il n'a pas déjà les données dans l'attributs (sauf si boolForcerUpdate est à true)

+ mVFunUpdatemLstObjCJsonMeteoMoyenneTemperatureAir(dtmDateDeDebut : DateTime, dtmDateDeFin : DateTime, boolForcerUpdate = false : bool) : void

Met à jour mLstObjCJsonMeteoMoyenneTemperatureAir si il n'a pas déjà les données dans l'attributs (sauf si boolForcerUpdate est à true)

+ mVFunUpdatemLstObjCJsonMeteoMoyenneVitesseVent(dtmDateDeDebut : DateTime, dtmDateDeFin : DateTime, boolForcerUpdate = false : bool) : void

Met à jour mLstObjCJsonMeteoMoyenneVitesseVent si il n'a pas déjà les données dans l'attributs (sauf si boolForcerUpdate est à true)

+ mVFunUpdatemLstObjCJsonMeteo(dtmDateDeDebut : DateTime, dtmDateDeFin : DateTime, boolForcerUpdate = false : bool) : void

Met à jour mLstObjCJsonMeteoMoyenneCJsonMeteo si il n'a pas déjà les données dans l'attributs (sauf si boolForcerUpdate est à true), met aussi à jour toute ses autres listes CJsonMeteo et utilise la méthode mFltFunCalculEvapotranspiration.

- mVFunVerifiermLstObjCJsonMeteoMoyenne

```
private void mVFunVerifiermLstObjMoyenne(  
lstObjCJsonAVerifier : List<CJsonMeteo>, fltValMin : float, fltValMax : float, shtEcartJournalier : short  
)
```

lstObjCJsonAVerifier : Liste à vérifier

fltValMin : Valeur minimale acceptée

fltValMax : Valeur maximale acceptée

shtEcartJournalier : Ecart de valeur d'un jour à l'autre accepté

La méthode vérifie la logique des données, c'est à dire qu'elle doit repérer au mieux les valeurs anormales
La première manière de repérer une valeur anormale est si celle ci a une valeur trop grande ou trop petite

On doit donc vérifier que toute les valeurs soient entre fltValMin et fltValMax

Si ce n'est pas le cas, elle prend la valeur la plus proche entre fltValMin et fltValMax

La deuxième manière est d'observer la continuité des valeurs les unes après les autres

Si on a un pic soudain dans les valeurs, ça signifie sûrement que celle si est éronnée

Une valeur anormale peut être remarquée par ces valeurs environnantes, elle est anormalement haute ou basse comparée au reste

L'idéal est de pouvoir la comparer avec avec celle du jour d'avant et d'après

Cependant, il faut d'abord vérifier si la liste est assez longue pour le permettre
Il faut une longueur de au moins 3 éléments

Si jamais la valeur est en effet anormale, alors elle devient la moyenne entre la valeur du jour d'avant et celle d'après

11 – Classe ApplicationHippodrome

Classe héritant de Form, elle sert à l'affichage de l'application et a en composition directement ou indirectement toutes les classes stipulées plus haut

11.1 - Attributs de ApplicationHippodrome

11.1.1 - Attributs TabPage, TabControl et ToolTip

- mTbpConditionMeteo : TabPage

TabPage ayant des contrôles utilisateurs à propos des conditions de la météo

- mTbp : TabPage

TabPage ayant des contrôles utilisateurs à propos des conditions du terrain

- mTbp : TabPage

TabPage ayant des contrôles utilisateurs à propos de l'état du terrain

- : TabControl

Tab qui inclut mTbpConditionMeteo, mTbp et mTbp

- mTltToolTipGlobal : ToolTip

ToolTip qui va donner des informations supplémentaires sur la majorité des contrôles utilisateurs

11.1.2 - Attributs CApi, CInfoMeteo, CInfoTerrain et CArrosage

- mObjCApi : CApi

Objet CApi qui va communiquer avec l'API

- mObjCDirectionVent : CDirectionVent

Objet CDirectionVent en association avec mObjCApi. Il va pouvoir renseigner les informations sur la direction du vent et va être utilisé par mLblDirectionVent

- mObjCPluviometrie : CPluviometrie

Objet CPluviometrie en association avec mObjCApi. Il va pouvoir renseigner les informations sur la pluviométrie et va être utilisé par mLblPluviometrie et mChtMeteo

- mObjCHygrometrie : CHygrometrie

Objet CHygrometrie en association avec mObjCApi. Il va pouvoir renseigner les informations sur l'hygrométrie et va être utilisé par mLblHygrometrie et, mChtMeteo et mObjCEvapotranspiration

- mObjCVitesseVent : CVitesseVent

Objet CVitesseVent en association avec mObjCApi. Il va pouvoir renseigner les informations sur la vitesse du vent et va être utilisé par mLblVitesseVent, mChtMeteo et mObjCEvapotranspiration

- mObjCTemperatureAir : CTemperatureAir

Objet CTemperatureAir en association avec mObjCApi. Il va pouvoir renseigner les informations sur la température de l'air et va être utilisé par mLblTemperatureAir, mChtMeteo et mObjCEvapotranspiration

- mObjCEnsoleillement : CEnsoleillement

Objet CEnsoleillement en association avec mObjCApi. Il va pouvoir renseigner les informations sur l'ensoleillement et va être utilisé mObjCEvapotranspiration

- mObjCEvapotranspiration : CEvapotranspiration

Objet CEvapotranspiration. Il va pouvoir renseigner les informations sur l'évapotranspiration et va être utilisé par mLblEvapotranspiration et mChtMeteo. Il est en association avec mObjCApi, mObjCEnsoleillement, mObjCTemperatureAir, mObjCVitesseVent et mObjCHygrometrie

11.1.3 - Label

- mLblDirectionVent : Label

Label affichant la donnée la plus récente sur la direction du vent

- mLblPluviometrie : Label

Label affichant la donnée la plus récente sur la pluviométrie

- mLblHygrometrie : Label

Label affichant la donnée la plus récente sur l'hygrométrie

- mLblVitesseVent : Label

Label affichant la donnée la plus récente sur la vitesse du vent

- mLblTemperatureAir : Label

Label affichant la donnée la plus récente sur la température de l'air

- mLblEvapotranspiration : Label

Label affichant la donnée la plus récente sur l'évapotranspiration

11.1.4 - Attributs Autre

- mChtMeteo : Forms.DataVisualization.Charting.Chart

Chart affichant une ou plusieurs courbes. Les courbes à afficher sont définie par l'utilisateur via mCbbSelectionValeur, leur période sont définie par l'utilisateur via mDtmpDateDeDebut mDtmpDateDeFin et mBtnConfirmationDate

- mCbbSelectionValeur : ComboBox

ComboBox permettant de sélectionner les différentes valeur à afficher sur mChtMeteo

- mDtmpDateDeDebut : DateTimePicker

Calendrier permettant de sélectionner la date de la données la plus ancienne souhaitée sur les courbes de mChtMeteo

- mDtmpDateDeFin : DateTimePicker

Calendrier permettant de sélectionner la date de la données la plus récente souhaitée sur les courbes de mChtMeteo

- mBtnConfirmationDate : Button

Bouton permettant d'actualiser la période des courbes de mChtMeteo avec les dates de mDtmpDateDeDebut et mDtmpDateDeFin

- mGpbGroupBoxChart : GroupBox

GroupBox regroupant mChtMeteo et mCbbSelectionValeur

mLblAjouterDateArrosage

mLblDateDernierArrosage
mLblDateDernierArrosage
PisteCirculaire mLblDate
DernierArrosagePisteDroite

mTabConditionMeteo